We all have experienced a scenario in which we are left with no other option but to click on a link so that the particular Web page gets downloaded.

What should be considered before creating a Web robot

Before developing a Web robot, we need to carefully analyse the task we want it to perform. Here are a few general considerations that need to the looked at before developing a Web robot.

1. Analysis of the task or set of tasks that we want Web robots to perform for us. It's good to use Web bots for repetitive tasks or anything that takes a lot of time when done manually, compared to when it's done automatically.
2. The sequence of steps that the Web robot needs to take so that the task is completed in the minimum amount of time needs to be well planned. It helps to obtain an efficient, error-free and effective program.
3. Check if the Web bot requires manual intervention or if the task can be fully automated.
4. Properly analyse the scripting or programming language that best suits the development of the Web bot, which can perform the specific task in the least time and with minimum cost and effort.
5. Check if the Web bot's requests, edits or any other actions need to be logged.
6. Check if the Web bot needs to run inside a Web browser or just be a standalone program.
7. If the Web bot runs on a remote server, check if it is possible for other editors to operate it.

Developing a small Web bot, free of cost

After we have made our considerations, we are good to go ahead with the development of the Web bot. Let's develop a small one, using Python scripting in order to scan a specific Web page.

You need to have the following installed on your system:

- Python 3.x or above
- Any text-editing application such as Notepad, etc

Now follow these steps:

- Open any plain text editing application, like Notepad, which is included with Microsoft Windows, where a Python script for the Web robot application can be written.
- Initiate the Python script by including the lines of code given below; the reason for including specific lines in the code has also been mentioned as 'comments'.

```
# importing required library files
 from html.parser import HTMLParser
 from urllib.request import urlopen
 from urllib import parse
# Creating a class called LinkParser that inherits some
methods from HTMLParser
 class LinkParser(HTMLParser):
# adding some more functionality to HTMLParser
 def handle_starttag(self, tag, attrs):
# checking for the beginning of a link as they are normally
present in <a> tag
       if tag == 'a':
            for (key, value) in attrs:
                if key == 'href':
# grabbing new URL and adding the base URL to it so that it
can be added to collection of links
                    newUrl = parse.urljoin(self.baseUrl,
value)
                    self.links = self.links + [newUrl]
# Function to get links that spider() function will call
    def getLinks(self, url):
        self.links = []
        self.baseUrl = url
# Using the urlopen function from the standard Python 3
library
        response = urlopen(url)
        if response.getheader('Content-Type')=='text/html':
            htmlBytes = response.read()
            htmlString = htmlBytes.decode("utf-8")
            self.feed(htmlString)
            return htmlString, self.links
        else:
            return "",[]
# spider() function which takes in an URL, a word to find, and
no. of pages to search through
        def spider(url, word, maxPages):
            pagesToVisit = [url]
            numberVisited = 0
            foundWord = False
 # Creating a LinkParser to get all links on the page.
    while numberVisited < maxPages and pagesToVisit != [] and
not foundWord:
        numberVisited = numberVisited +1
 # starting from the first page of the collection:
        url = pagesToVisit[0]
        pagesToVisit = pagesToVisit[1:]
        try:
            print(numberVisited, "Visiting:", url)
            parser = LinkParser()
            data, links = parser.getLinks(url)
            if data.find(word)>-1:
                foundWord = True
                pagesToVisit = pagesToVisit + links
                print(" **Success!**")
        except:
            print(" **Failed!**")
    if foundWord:
        print("The word", word, "was found at", url)
    else:
        print("Word never found")
```